

Paterni ponašanja

Strategy patern

Strategy patern izdvaja algoritam iz matične klase i uključuje ga u posebne klase. U našem sistemu može se koristiti za implementaciju različitih načina pretrage knjiga. Na primjer, moguće je definisati strategije pretrage po nazivu knjige, autoru, ISBN broju ili kategoriji. Klasa *Knjiga* ili poseban servis za pretragu koristio bi odgovarajuću strategiju u zavisnosti od zahtjeva korisnika.

Patern se može koristiti i za različite metode obračuna kašnjenja prilikom vraćanja knjiga. Biblioteka može imati različita pravila obračuna za različite klijente. Prednost ovog pristupa je jednostavno dodavanje novih algoritama bez izmjene postojećeg koda.

State patern

State patern mijenja način ponašanja objekata na osnovu trenutnog stanja. Umjesto velikog broja naredbi, svako stanje se implementira kao zasebna klasa.

U našem sistemu ovaj patern se može primjeniti na klase *Posudba*, *ZahtjevZaPosudbu*, *NabavkaKnjiga* i *Knjiga*. Na primjer, objekat *Posudba* može imati stanje *aktivna*, *vracena* i *kasnjenje*. Svako stanje implementira različita pravila ponašanja. Aktivna posudba omogućava vraćanje knjige, dok vraćena posudba ne dozvoljava izmjene.

Na isti način mogu se implementirati stanja zahtjeva za posudbu (*na_cekaju*, *odobren*, *odbijen*) i stanja nabavke knjige (*u_obrađi*, *poslano*, *primljeno*, *otkazano*). Primjenom ovog paterna smanjuje se broj uslovnih grananja u kodu i olakšava dodavanje novih stanja u budućnosti.

Template Method patern

Template Method patern omogućava algoritmima da izdvoje pojedine korake u podklase. U našem sistemu može se koristiti za obradu različitih vrsta zahtjeva. Svaki zahtjev prolazi kroz korake validacije, obrade i završetka, ali konkretna implementacija zavisi od vrste zahtjeva.

Na primjer, zahtjev za posudbu i zahtjev za nabavku knjige mogu koristiti istu osnovnu proceduru, ali sa različitim detaljima implementacije. Ovim se izbjegava dupliciranje koda i povećava mogućnost ponovne upotrebe koda.

Chain of Responsibility patern

Chain of Responsibility patern predstavlja listu objekata, ukoliko objekat ne može da odgovori prosljeđuje zahtjev narednom u nizu. U našem sistemu ovaj patern može biti korišten prilikom odobravanja zahtjeva za posudbu. Prvo se provjerava validnost članske kartice, zatim dostupnost knjige, nakon toga eventualna dugovanja korisnika i na kraju odobrenje od strane bibliotekara.

Svaki korak predstavlja zaseban objekat koji obrađuje dio zahtjeva. Ako je uslov zadovoljen, zahtjev se proslijeđuje sljedećem objektu u lancu. Ovakav pristup omogućava jednostavno dodavanje novih provjera bez izmejen postojećeg procesa.

Command patern

Command patern pretvara zahtjev u samostalni objekat, omogućavajući paternizaciju klijenata različitim zahtjevima, njihovo stavljanje u red čekanja (queue), logovanje, kao i podršku za operacije poništavanja (undo/redo).

U našem sistemu, operacije kao što su odobravanje zahtjeva, odbijanje zahtjeva, vraćanje knjige, produženje članstva i kreiranje narudžbe mogu biti implementirane kao komande. Na primjer, komanda `OdobriZahtjevCommand` sadrži svu logiku potrebnu za odobravanje zahtjeva za posudbu. Sistem može čuvati historiju izvršenih komandi i omogućiti poništavanje određenih akcija.

Iterator patern

Iterator patern omogućava pristup elementima kolekcije sekvencijalno bez poznavanja interne strukture kolekcije. U našem sistemu može se koristiti za prolazak kroz listu knjiga, korisnika, aktivnih posudbi ili zahtjeva za posudbu.

Na primjer, administrator može prolaziti kroz kompletnu kolekciju knjiga koristeći iterator bez direktnog pristupa strukturi u kojoj su podaci pohranjeni.

Mediator patern

Mediator patern enkapsulira protokol za komunikaciju među objektima dozvoljavajući da objekti komuniciraju bez međusobnog poznavanja interne strukture. U našem sistemu veliki broj klasa međusobno sarađuje. Posudba komunicira sa knjigom i korisnikom, zahtjevi za posudbu zavise od korisnika i knjiga, a nabvka zavisi od dobavljača.

Umjesto direktne komunikacije između svih klasa, moguće je uvesti centralni objekat *BibliotekaMediator* koji koordinira razmjenu informacija između njih.

Observer patern

Observer patern uspostavlja relaciju između objekata takvu da kad se stanje jednog objekta promijeni svi vezani objekti dobiju informaciju. U našem sistemu može se koristiti za slanje obavještenja korisnicima. Kada se promijeni status zahtjeva za posudbu, korisnik automatski dobije obavještenje.

Visitor patern

Visitor patern definiše i izvršava nove operacije nad elementima postojeće strukture ne mijenjajući samu strukturu. U našem sistemu može se koristiti za generisanje izvještaja. Jedan visitor može računati statistiku posudbi, drugi broj aktivnih članova, a treći informacije o nabavkama knjiga.

Interpreter patern

Interpreter patern podržava interpretaciju instrukcija napisanih za određenu upotrebu. U našem sistemu može biti primijenjen za naprednu pretragu knjiga. Korisnik bi mogao unositi složene upite, na primjer. autor="Andrić" AND kategorija="Roman". Sistem bi interpretirao uneseni izraz i generisao odgovarajuću pretragu baze podataka.

Momento patern

Momento patern omogućava spašavanje internog stanja nekog objekta van sistema i njegovo ponovno vraćanje. U našem sistemu može se koristiti za čuvanje podataka korisnika, knjiga ili zahtjeva za posudbu. Na primjer, ukoliko bibliotekar greškom promijeni podatke o knjizi, sistem može vratiti prethodno stanje koristeći sačuvani momento objekat.